CPSC 313: Computer Hardware and Operating Systems

Unit 2: Pipelining

Branch Prediction

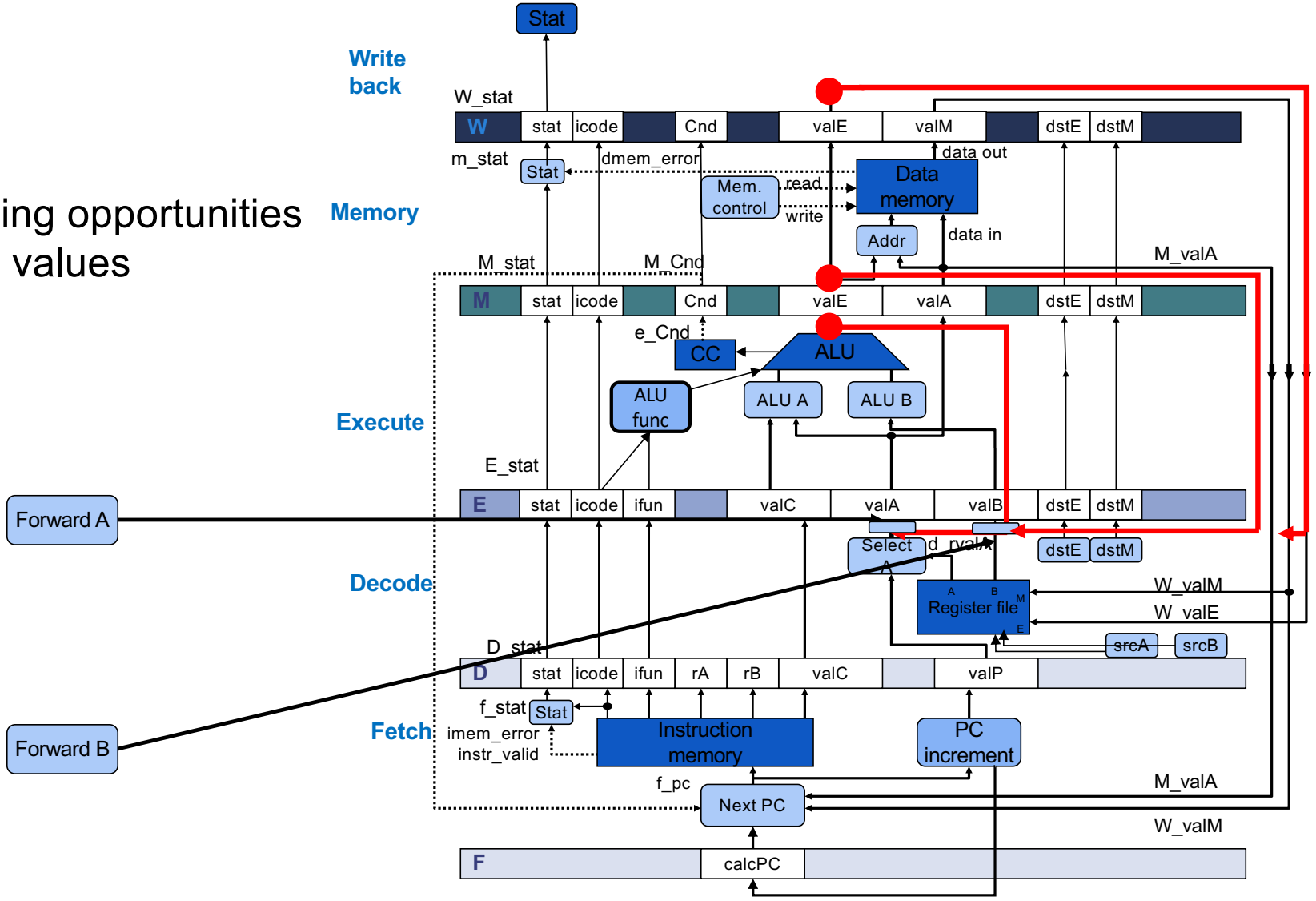
Admin

- Quiz 2:
  - Next week, please ensure you've reserved your slot.
  - Quiz 2 information and practice are available now.
- Labs
  - Lab 4 is due on Sunday
  - Lab 5 will become accessible at 17:00 Friday.

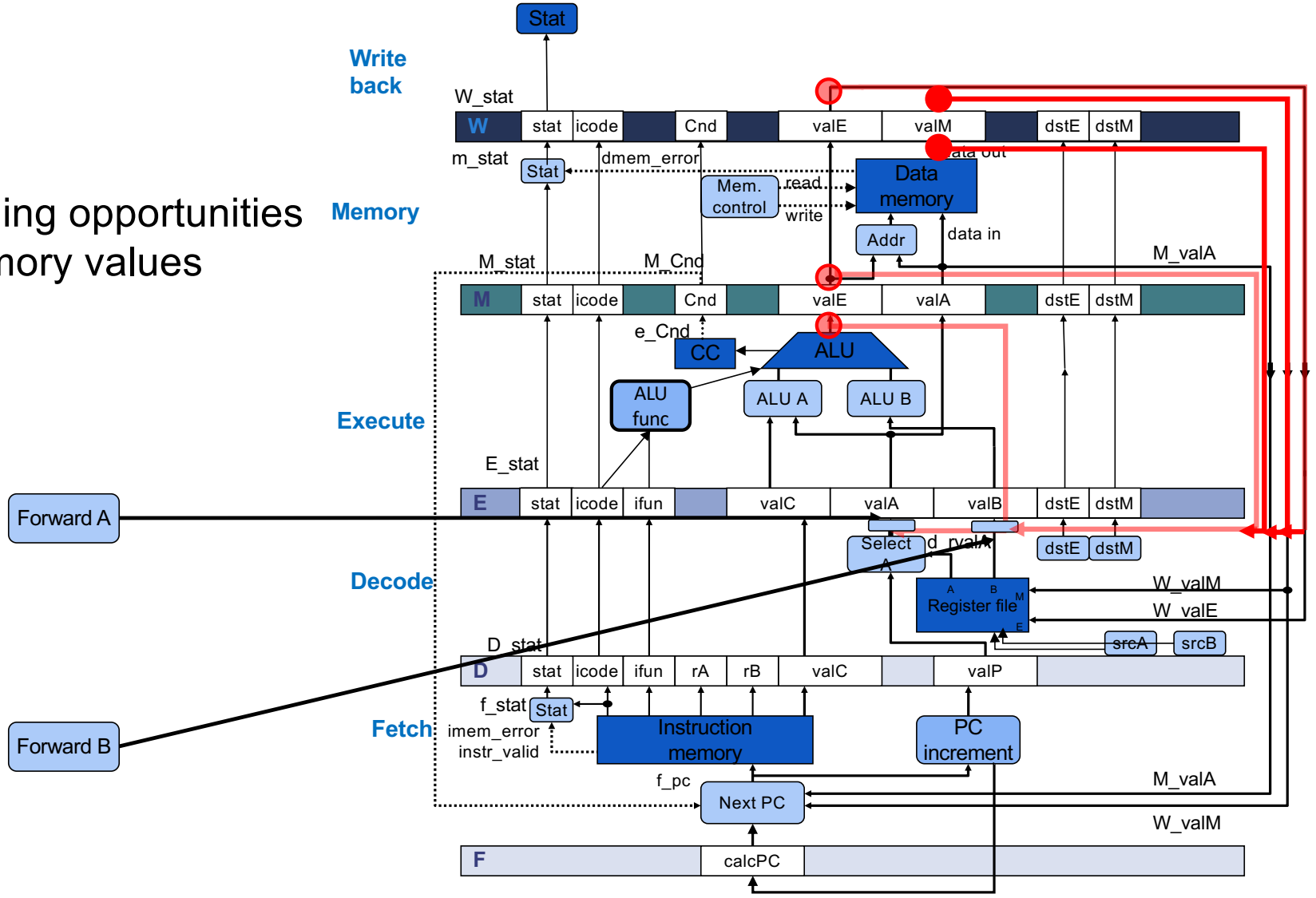
Today

- Learning outcomes
  - Describe what branch prediction is
  - Enumerate different branch prediction algorithms
  - Exploit branch prediction to improve CPI

- Recall:
  - forwarding opportunities for ALU values



- Recall:
  - forwarding opportunities for Memory values



Dealing with Control Hazards

- Can you tell me how much we will stall?
  - jmp and call:
  - ret:
  - jxx:

Dealing with Control Hazards

- Can you tell me how much we will stall?
  - jmp and call: no stalls, we know where the next instruction is in Fetch.
  - ret:
  - jxx:

Dealing with Control Hazards

- Can you tell me how much we will stall?
  - jmp and call: no stalls, we know where the next instruction is in Fetch.
  - ret: three stalls (we have to wait for `ret` to have completed Memory).
  - jxx:

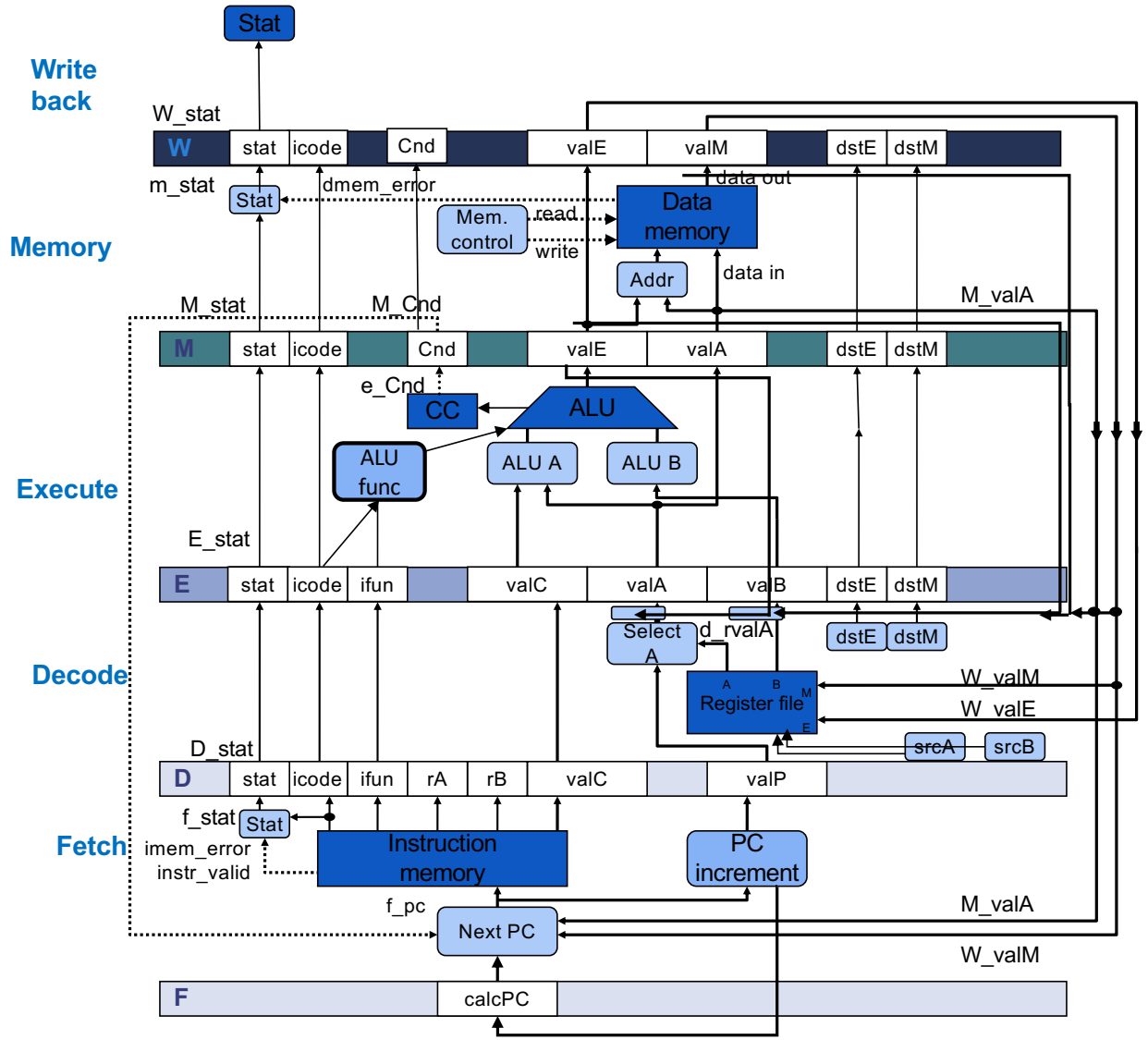
Dealing with Control Hazards

- Can you tell me how much we will stall?
  - jmp and call: no stalls, we know where the next instruction is in Fetch.
  - ret: three stalls (we have to wait for `ret` to have completed Memory).
  - jxx: we can guess

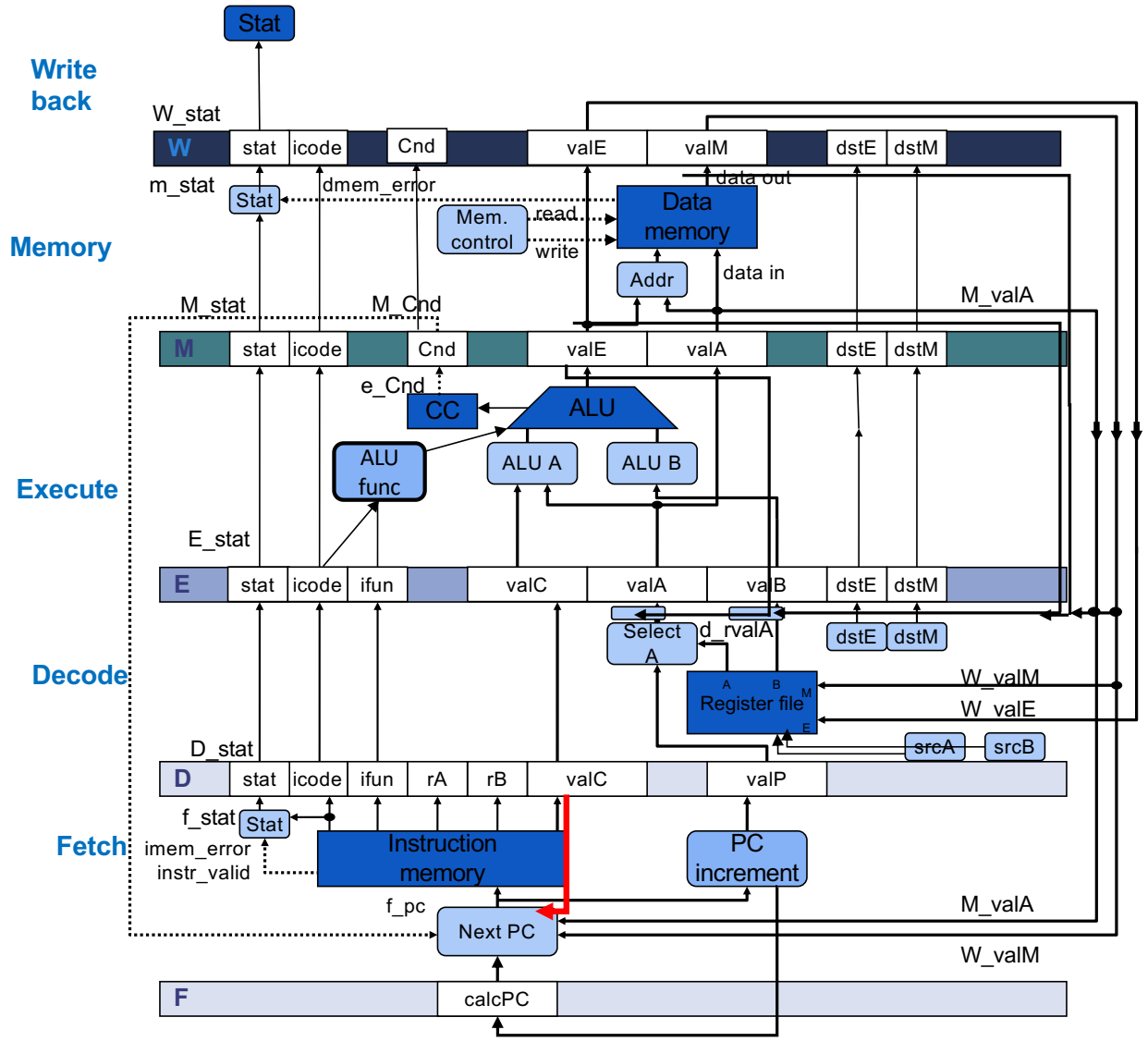
Dealing with Control Hazards

- Can you tell me how much we will stall?
  - jmp and call: no stalls, we know where the next instruction is in Fetch.
  - ret: three stalls (we have to wait for `ret` to have completed Memory).
  - jxx: we can guess
    - Never taken (assume that the nextPC is going to be valP)
    - Always taken (assume that the nextPC is going to be valC)
    - Or fancier things, but how should we decide which is correct?
  - Let's try some examples.

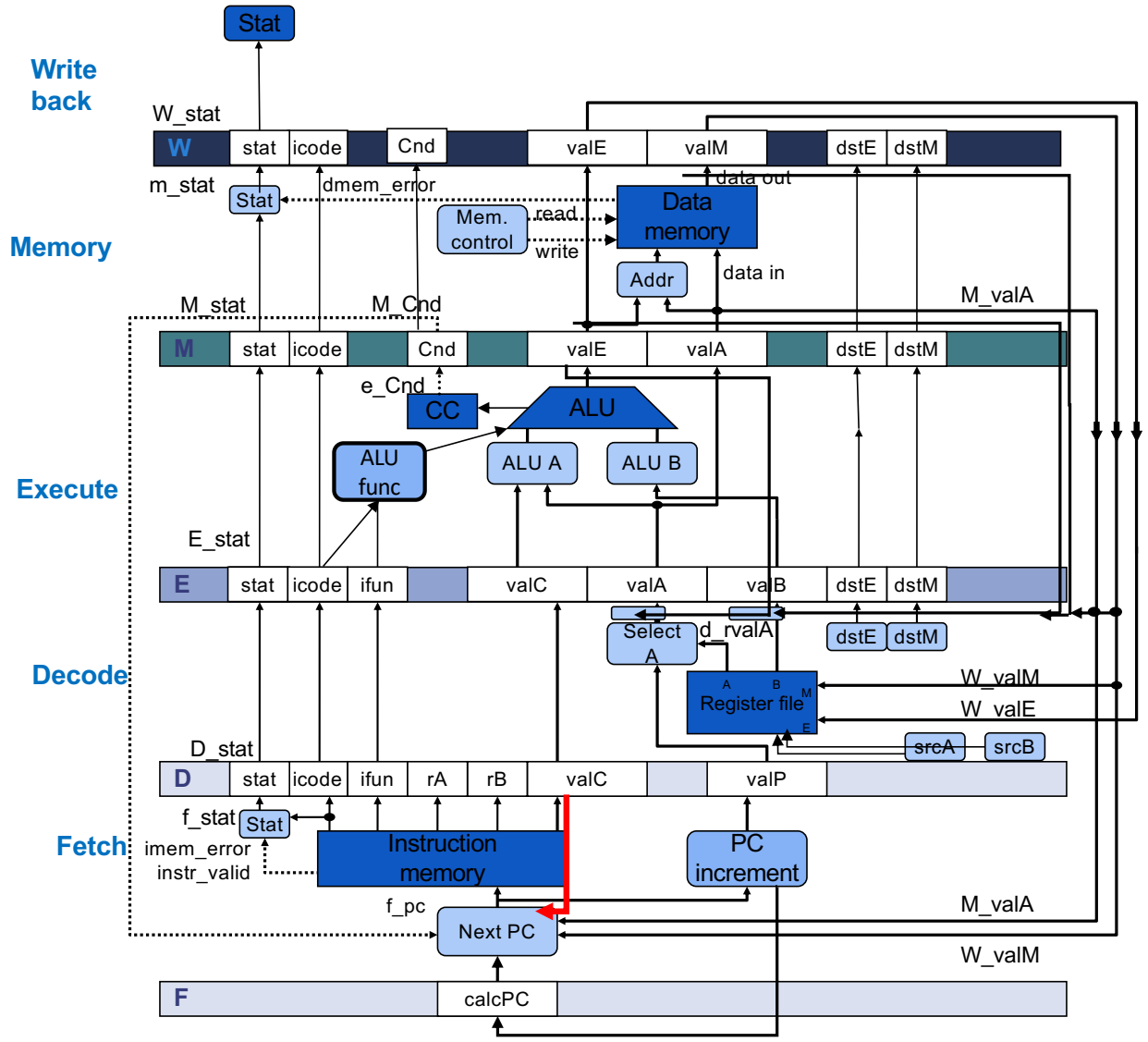
- Branch prediction:
  - ALWAYS TAKEN
- Do we have what we need when we need it to predict this?



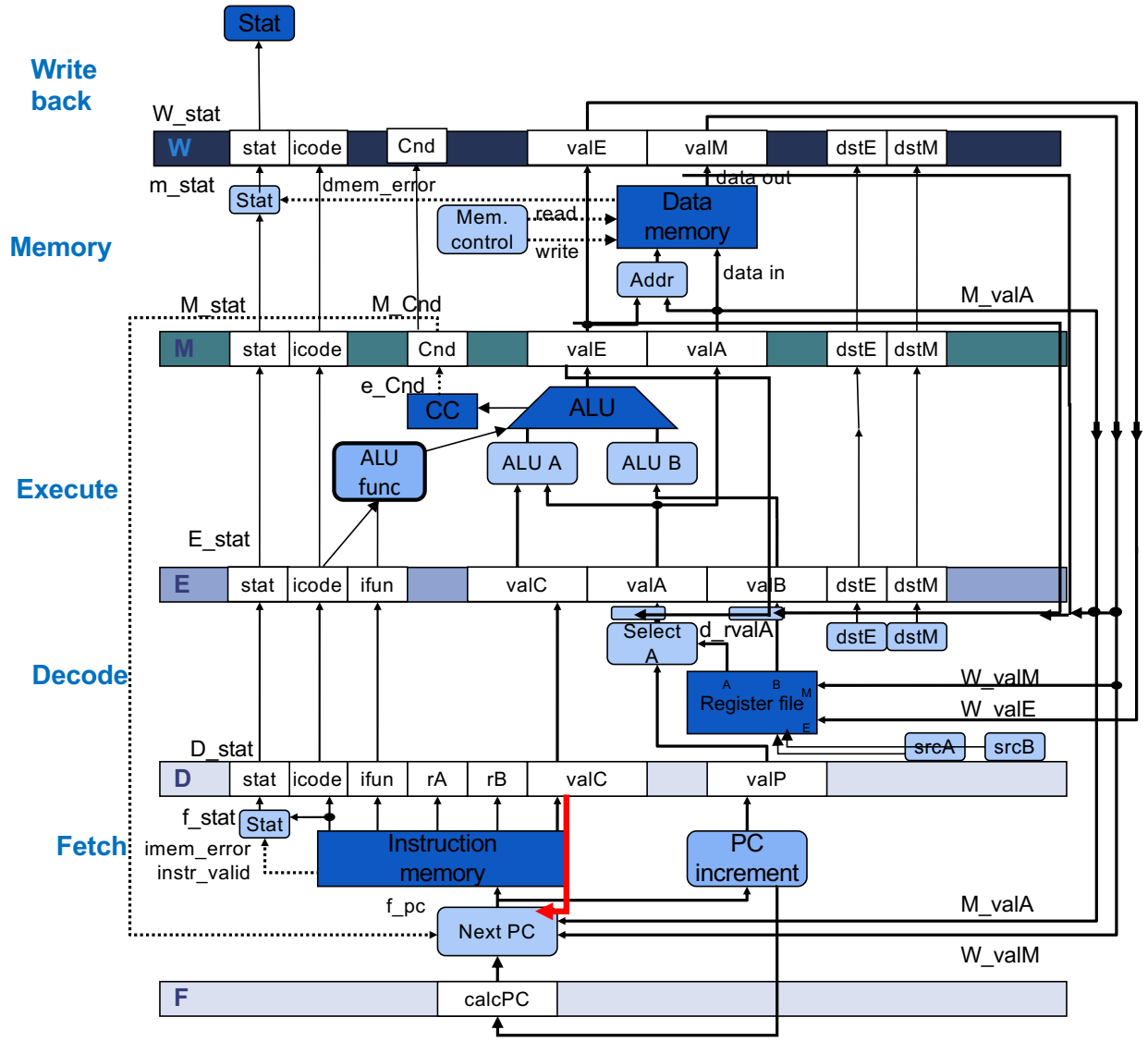
- Branch prediction:
  - ALWAYS TAKEN
- Do we have what we need when we need it to predict this?
  - Yes



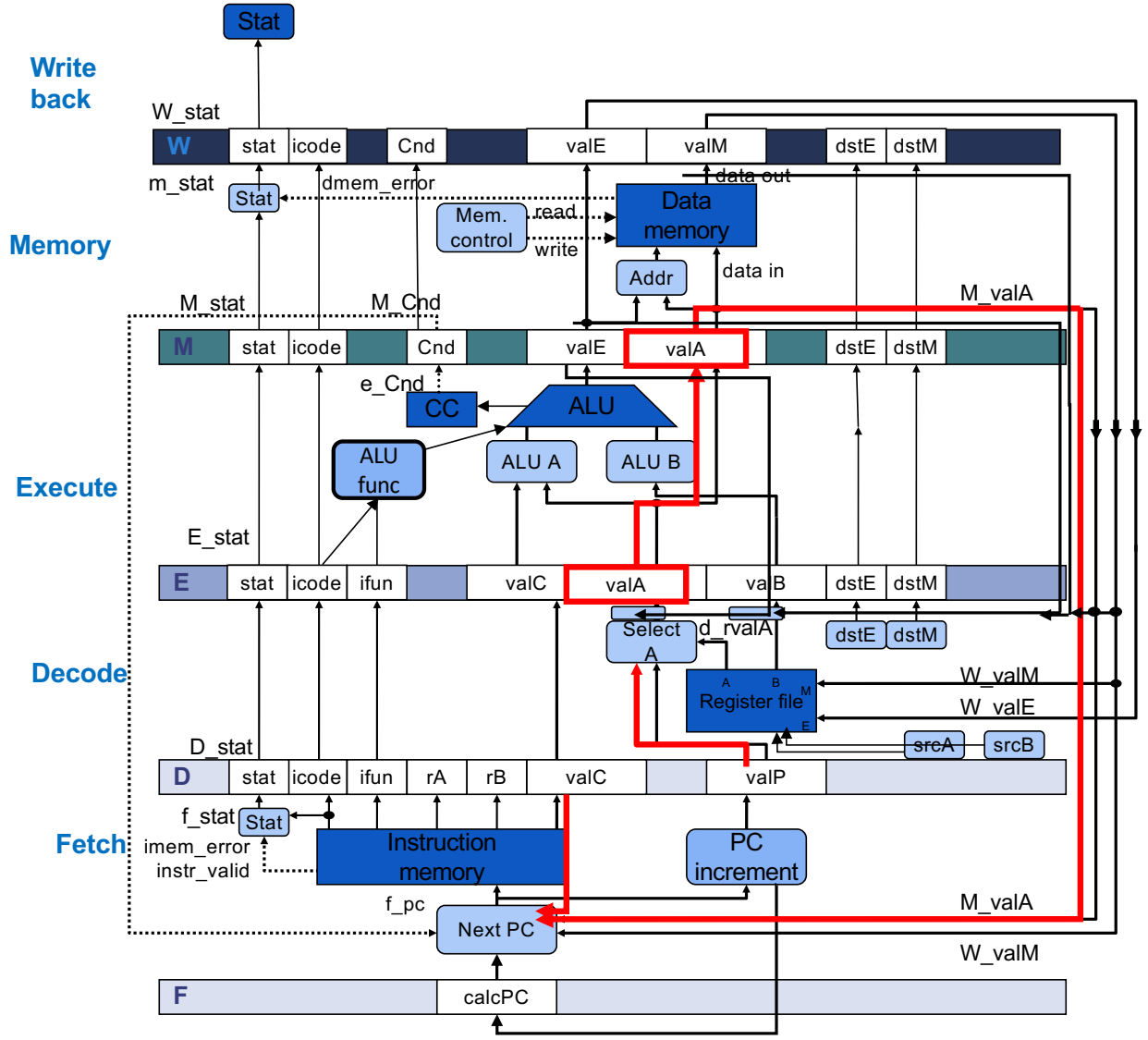
- Branch prediction:
  - ALWAYS TAKEN
- Do we have what we need when we need it to predict this?
  - Yes
- What if we mispredict?



- Branch prediction:
  - ALWAYS TAKEN
- Do we have what we need when we need it to predict this?
  - Yes
- What if we mispredict?
  - When do we realize it?



- Branch prediction:
  - ALWAYS TAKEN
- Do we have what we need when we need it to predict this?
  - Yes
- What if we mispredict?
  - When do we realize it?
  - We need to keep track of `valP`.



Branch Prediction

- Let's convert these two code snippets into y86

```
j = 1;
do {
    j++;
} while j < 10;
```

```
if (p == NULL) {
    //handle error
    return -1;
}
```

Branch Prediction

- Let's convert these two code snippets into y86

```
j = 1;
do {
    j++;
} while j < 10;

# Let's assume j is in %rsi
       irmovq 1, %rax       # incr = 1
       irmovq 1, %rsi       # j = 1

Loop:  addq %rax, %rsi
       irmovq 10, %rdi      # rdi = 10
       subq %rsi, %rdi      # set CC
       jg Loop
```

```
if (p == NULL) {
    //handle error
    return -1;
}

# Let's assume p is in %rbp
# and is already assigned a value
       andq %rbp, %rbp
       je error
       # The rest of the function
       # goes here

error: # Handle the 'if' clause
       irmovq  0xFFFFFFFFFFFFFFFF, %rax
       ret
```

Branch Prediction

```
# Let's assume j is in %rsi
        irmovq 1, %rax # incr = 1
        irmovq 1, %rsi # j = 1

Loop:   addq %rax, %rsi
        irmovq 10, %rdi         # rdi = 10
        subq %rsi, %rdi         # set CC
        jg Loop
```

```
# Let's assume p is in %rbp
# and is already assigned a value
        andq %rbp, %rbp
        je error
        # The rest of the function
        # goes here

error:  # Handle the 'if' clause
        irmovq  0xFFFFFFFFFFFFFFFF, %rax
        ret
```

- Observations:
  - Backwards jumps are often used to repeat a loop body, so they are usually taken.
  - Forward jumps are usually conditionals, and may or may not be taken.
- So a reasonable strategy is to assume backward jumps are taken, forward jumps are not.

Steering Branch Prediction (Linux Kernel)

- You can give hints
- Linux has two macros:

```
#define likely(x)   __builtin_expect(!!(x), 1)
#define unlikely(x) __builtin_expect(!!(x), 0)
```

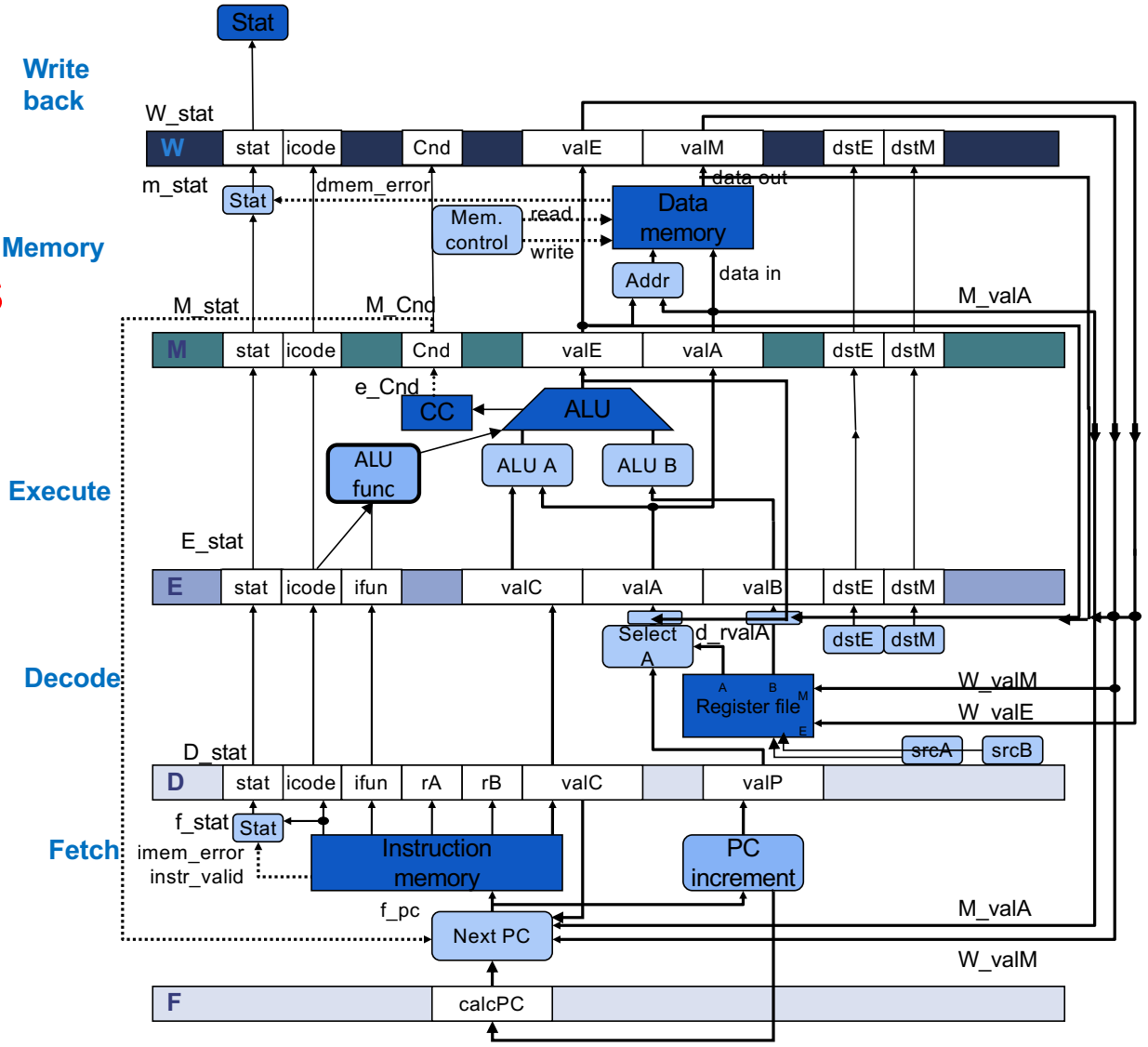
- For instance

```
if (unlikely(x)) foo();
```

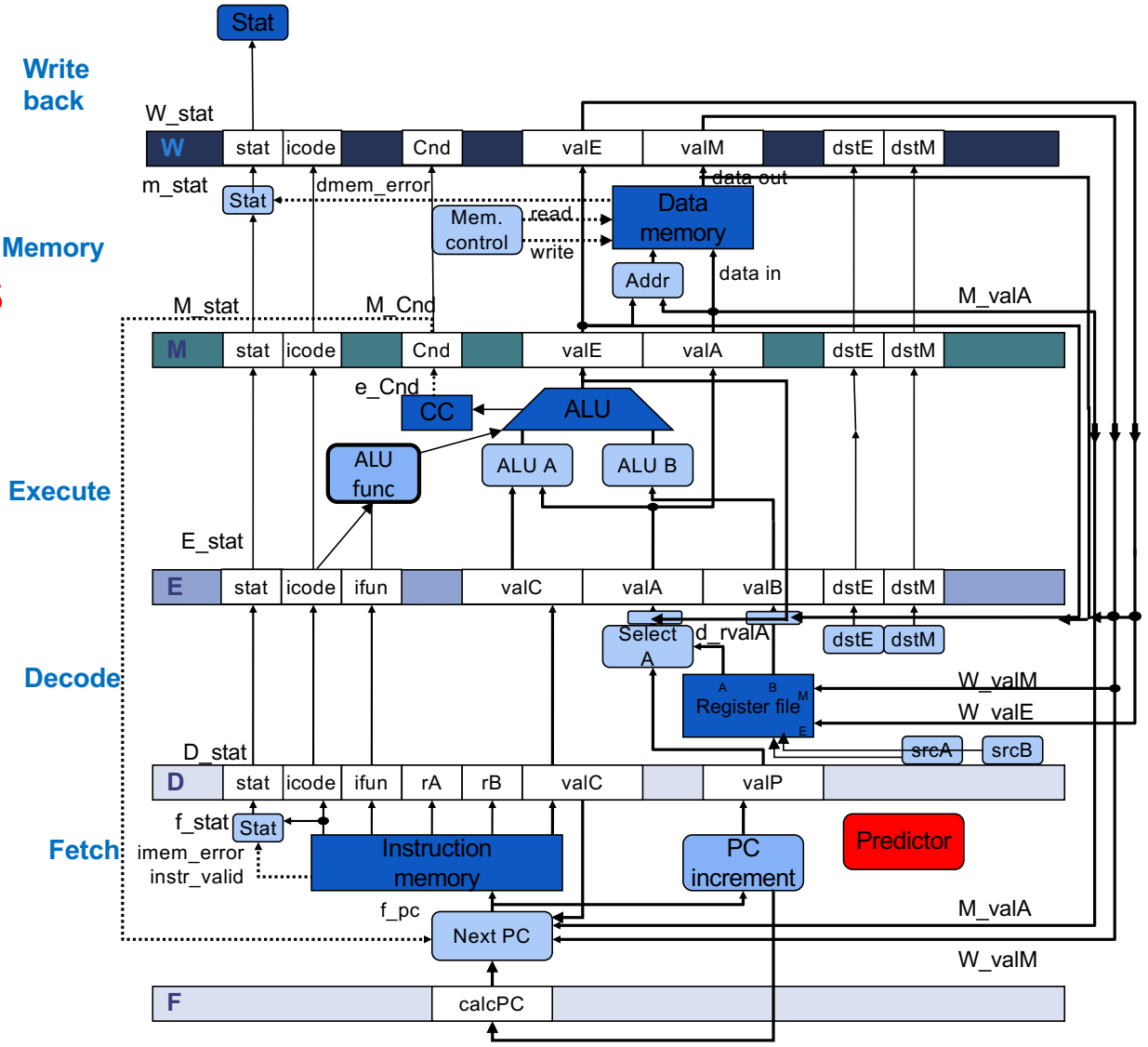
  would indicate we believe `x` is probably false and so we probably won't call `foo()`.

- Results may vary...

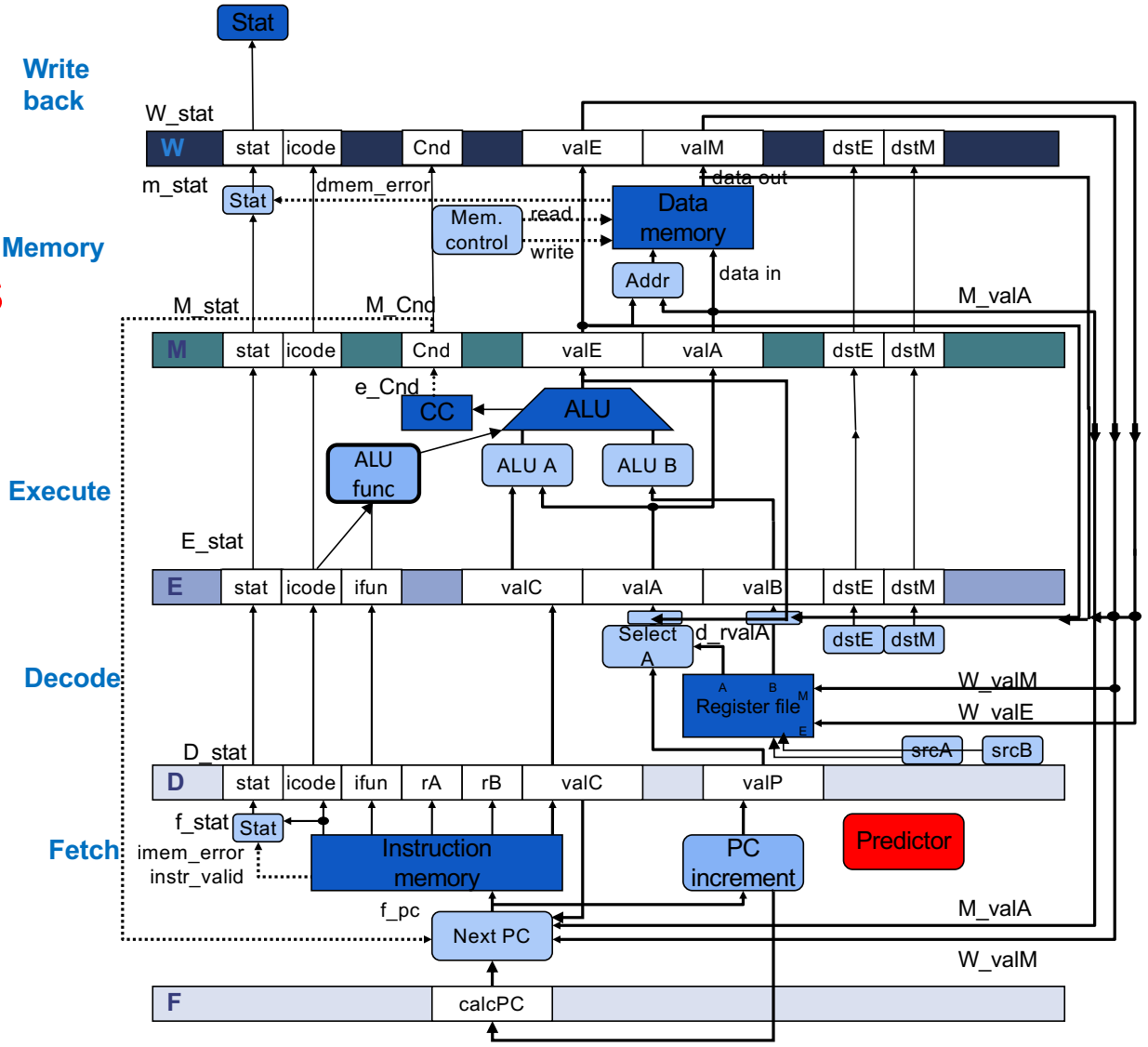
- Branch prediction:
  - IMPLEMENTING MORE COMPLEX STRATEGIES



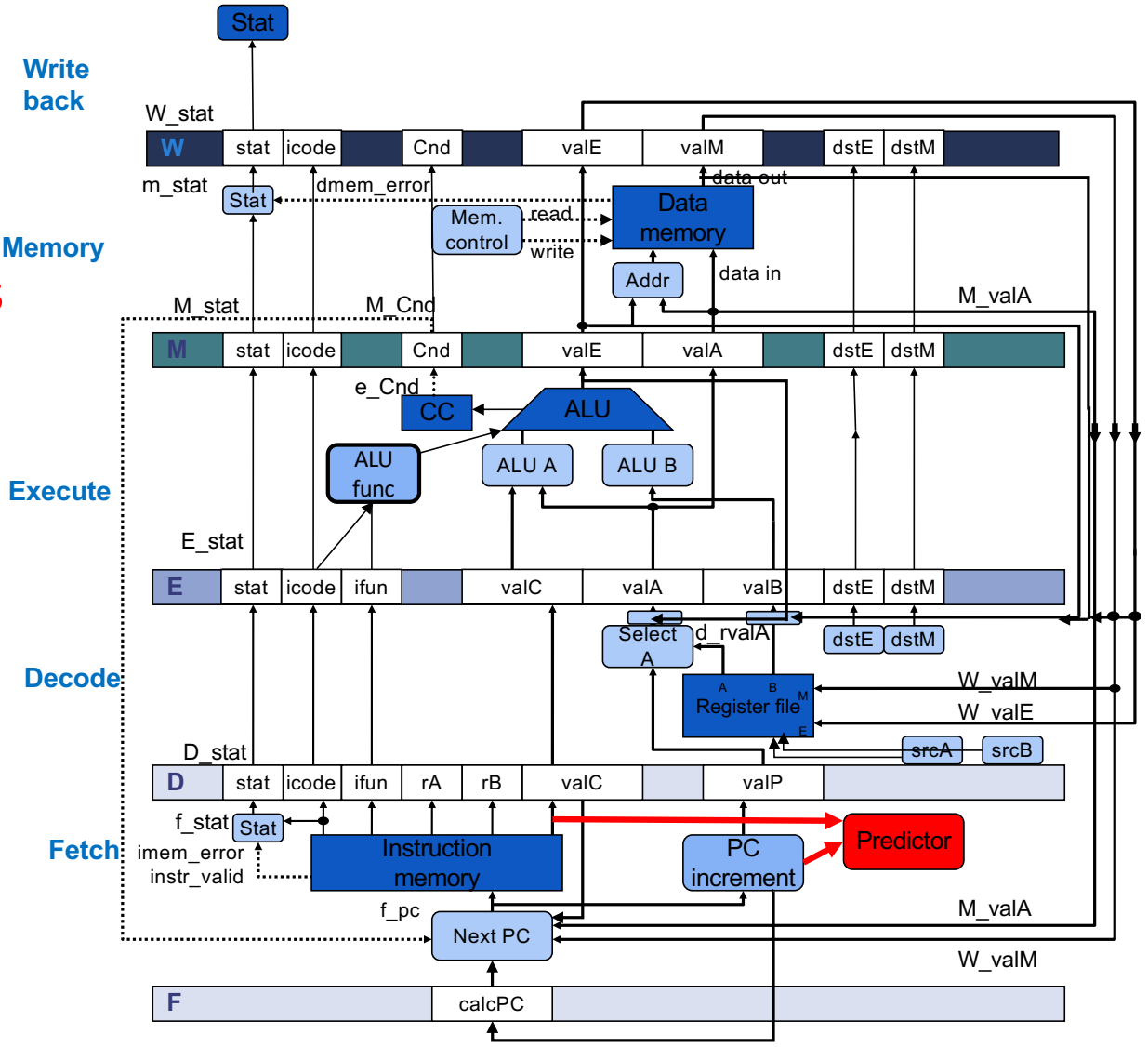
- Branch prediction:
  - IMPLEMENTING MORE COMPLEX STRATEGIES
- First we introduce prediction logic.



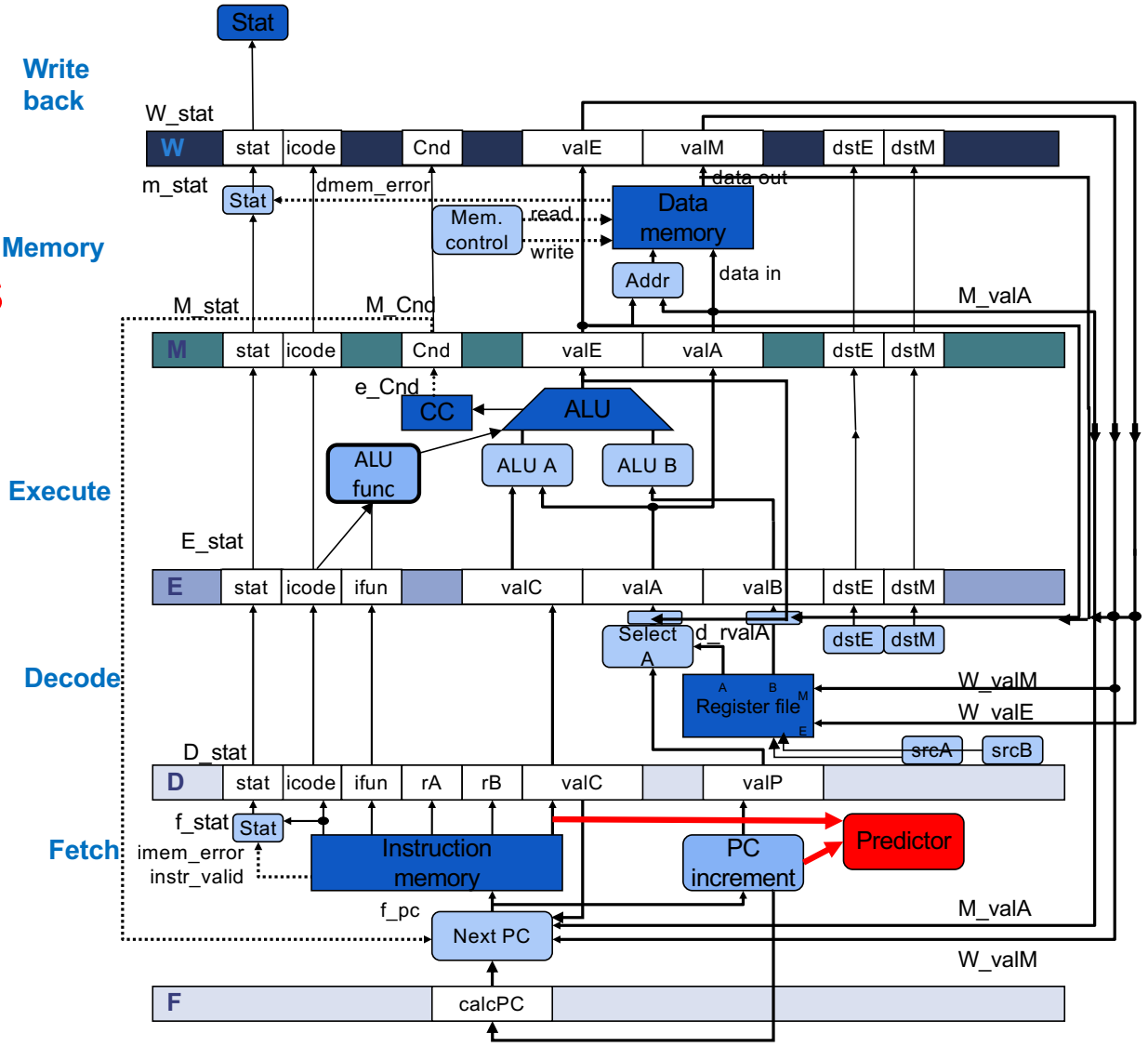
- Branch prediction:
  - IMPLEMENTING MORE COMPLEX STRATEGIES
- First we introduce prediction logic.
- What inputs does it need?



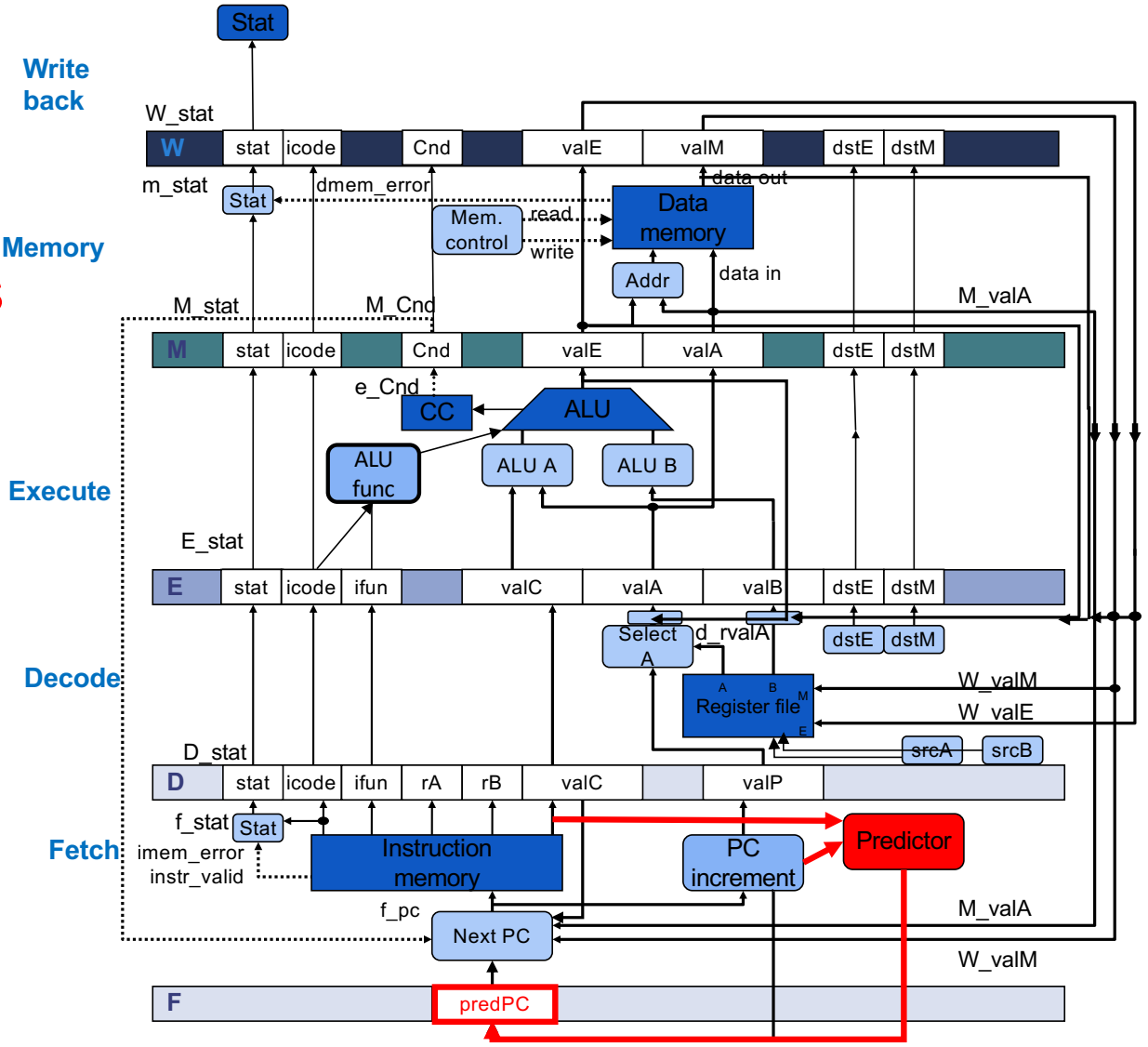
- Branch prediction:
  - IMPLEMENTING MORE COMPLEX STRATEGIES
- First we introduce prediction logic.
- What inputs does it need?



- Branch prediction:
  - IMPLEMENTING MORE COMPLEX STRATEGIES
- First we introduce prediction logic.
- What inputs does it need?
- And where does its output go?



- Branch prediction:
  - IMPLEMENTING MORE COMPLEX STRATEGIES
- First we introduce prediction logic.
- What inputs does it need?
- And where does its output go?



Fun Facts about Branch Prediction

- Always taken typically has about a 60% success rate
- Never taken typically has about a 40% success rate
- Some processors do:
  - Backward taken; forward not taken (65% success rate)

Fun Facts about Branch Prediction

- Some processors devote lots of fancy hardware to branch prediction
  - 1 bit: do whatever you did last time.
  - More bits for more complicated schemes.
  - Entries for branches at different values of the PC.
- Other processors let you execute instructions while you're waiting to determine what's going to happen with your jump (these instructions occupy what are known as branch delay slots).

Some Recent Research on Branch Prediction

- Half&Half: Demystifying Intel's Directional Branch Predictors for Fast, Secure Partitioned Execution
  by Hosein Yavarzadeh et al.

  https://ieeexplore.ieee.org/document/10179415/

- "This work presents the first exhaustive analysis of modern conditional branch prediction structures"

The Problem with Branch Prediction

- What happens if you are wrong?
- Consider:

Let's assume that our prediction algorithm is "never taken"

```
        irmovq 0x1, %rax
        irmovq 0x1, %rbx
        subq %rax, %rbx
        je skip
        irmovq 0x5000, %rcx
        irmovq 0x5000, %rdx
        # more instructions here
skip:
        addq %rcx, %rbx
        mulq %rbx, %rdx
```

Mis-Prediction (1)

- What happens if you are wrong?
- Consider:

```
        irmovq 0x1, %rax
        irmovq 0x1, %rbx
        subq %rax, %rbx
        je skip
        irmovq 0x5000, %rcx
        irmovq 0x5000, %rdx
        # more instructions here
skip:
        addq %rcx, %rbx
        mulq %rbx, %rdx
```

Let's assume that our prediction algorithm is "never taken"

You have two bad instructions in the pipeline!

- That is, instructions you should not have executed!
- So what do you do???

Mis-Prediction (2)

- What happens if you are wrong?
- Consider:

```
        irmovq 0x1, %rax
        irmovq 0x1, %rbx
        subq %rax, %rbx
        je skip
        irmovq 0x5000, %rcx
        irmovq 0x5000, %rdx
        # more instructions here
skip:
        addq %rcx, %rbx
        mulq %rbx, %rdx
```

Let's assume that our prediction algorithm is "never taken"

You have two bad instructions in the pipeline!

- That is, instructions you should not have executed!
- So what do you do???

You cancel or squash or quash them!

- Replace them by nops.

Mis-Prediction (2)

- What happens if you are wrong?
- Consider:

```
        irmovq 0x1, %rax
        irmovq 0x1, %rbx
        subq %rax, %rbx
        je skip
        irmovq 0x5000, %rcx
        irmovq 0x5000, %rdx
        # more instructions
skip:
        addq %rcx, %rbx
        mulq %rbx, %rdx
```

Let's assume that our prediction algorithm is "never taken"

You have two bad instructions in the pipeline!

- That is, instructions you should not have executed!
- So what do you do???

You cancel or squash or quash them!

- Replace them by nops.

So:

- In the best case, conditional branches incur no penalty
- In the worst case (where you mispredict), it's no worse than if you'd done no prediction

Branch Prediction and CPI

- A good branch predictor can dramatically improve your CPI (cycles per instruction).
- We'll give you lots of practice with that.

In-class Exercise

- Remember to do multiple instances of a problem if there is only one problem listed!
  - We guessed maybe 5, but it's good to practice as many as you have time for!

Wrapping Up

- Branch prediction (tries to) avoid losing cycles on every conditional jump.
- A good predictor will reduce a program's CPI.
- You should be able to:
  - Compute CPI for a simple program
  - Given an unoptimized program, rearrange it to improve CPI

Why We Love Systems

